

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA INGENIERÍA
CARRERA DE SOFTWARE (SOFT-R)

**Tecnologías de Bases de Datos Distribuidas:
NewSQL vs. NoSQL**

Análisis Técnico de MongoDB Atlas como Base de Datos Distribuida Orientada a Documentos

Actividad: FORM-IC-02 — Investigación Individual

Preparatoria para el Informe Técnico Individual (TA-IND-02) — Semana 11

Asignatura: Aplicaciones Distribuidas

Nivel: Séptimo Nivel

Paralelo: A

Modalidad: Individual

Carácter: Formativa

Docente: Guerrero Ulloa Gleiston Cicerón

Estudiante:

Vinueza Sanchez Harold Nicolas

Repositorio del proyecto:

[https:](https://github.com/Harold-Vinueza/FORM-IC-02-Investigaci-n-Individual-Tecnolog-as-de-BDD-Distribuidas-NewSQL-vs.-NoSQL)

[//github.com/Harold-Vinueza/FORM-IC-02-Investigaci-n-Individual-Tecnolog-as-de-BDD-Distribuidas-NewSQL-vs.-NoSQL](https://github.com/Harold-Vinueza/FORM-IC-02-Investigaci-n-Individual-Tecnolog-as-de-BDD-Distribuidas-NewSQL-vs.-NoSQL)

MongoDB Atlas como Tecnología de Base de Datos Distribuida: Modelo de Consistencia, Distribución de Datos y Aplicabilidad Práctica

Guerrero Ulloa Gleiston Cicerón

Ingeniería en Software, Universidad Técnica Estatal de Quevedo

Aplicaciones Distribuidas — Séptimo Nivel, Paralelo A

Resumen—El presente trabajo analiza a MongoDB Atlas como representante del paradigma NoSQL dentro del ecosistema de bases de datos distribuidas, en contraste con los motores relacionales tradicionales y con las soluciones NewSQL. Se describen sus mecanismos de distribución de datos mediante particionamiento horizontal (*sharding*) y conjuntos de réplicas, se clasifica su comportamiento de acuerdo con el teorema CAP, se presenta un caso de uso real documentado y se discute su pertinencia para proyectos académicos orientados a aplicaciones distribuidas.

Index Terms—MongoDB Atlas, NoSQL, teorema CAP, bases de datos distribuidas, *sharding*, consistencia eventual.

I. INTRODUCCIÓN

El crecimiento sostenido del volumen de datos generado por aplicaciones web, móviles y de Internet de las Cosas ha llevado a que los sistemas de bases de datos tradicionales, pensados para operar en un único nodo o en esquemas rígidamente normalizados, resulten insuficientes frente a exigencias de escalabilidad horizontal y disponibilidad continua [4]. Como respuesta a este escenario surgieron, por un lado, las bases de datos NoSQL, que sacrifican parte de las garantías transaccionales clásicas a cambio de flexibilidad de esquema y escalado sencillo, y por otro las soluciones NewSQL, que buscan conservar las propiedades ACID mientras distribuyen los datos entre múltiples nodos [6]. Dentro de este panorama, MongoDB Atlas se posiciona como uno de los servicios NoSQL gestionados más adoptados en la industria, al ofrecer un modelo orientado a documentos con capacidades nativas de distribución, replicación automática y escalado elástico en la nube [4]. El objetivo de este documento es examinar sus características de distribución, su ubicación dentro del teorema CAP, un caso real documentado de su aplicación y la pertinencia de adoptarlo en un proyecto propio de aplicaciones distribuidas.

II. CARACTERÍSTICAS DE DISTRIBUCIÓN DE DATOS

MongoDB Atlas organiza la distribución de la información mediante dos mecanismos complementarios: la **replicación** y el **particionamiento horizontal o *sharding***. Cada conjunto de réplicas (*replica set*) mantiene copias sincronizadas de los datos en distintos nodos, de modo que si el nodo primario falla, uno de los nodos secundarios asume automáticamente ese rol, garantizando continuidad operativa [9]. Para conjuntos de datos que superan la capacidad de un solo servidor, o cuando la tasa de lecturas y escrituras es demasiado alta, el

motor recurre al *sharding*: los documentos de una colección se distribuyen entre múltiples fragmentos (*shards*) en función de una **clave de fragmentación** (*shard key*), mientras que un enrutador de consultas (*mongos*) se encarga de dirigir cada operación hacia el fragmento correspondiente sin que la aplicación cliente perciba la complejidad subyacente [7], [9]. Un componente adicional, los servidores de configuración, conserva los metadatos sobre cómo están distribuidos los rangos de datos entre fragmentos, lo cual permite que el sistema reequilibre automáticamente la carga a medida que el volumen de información crece [9]. La elección de una buena clave de fragmentación resulta determinante, pues una distribución desigual puede generar puntos calientes que concentran el tráfico en pocos nodos, deteriorando el rendimiento general del clúster [4].

III. MODELO DE CONSISTENCIA SEGÚN EL TEOREMA CAP

El teorema CAP, formulado inicialmente por Brewer y demostrado formalmente por Gilbert y Lynch, establece que ningún sistema distribuido puede garantizar simultáneamente consistencia, disponibilidad y tolerancia a particiones de red; ante una partición real, el diseñador debe optar por sacrificar una de las dos primeras propiedades [1], [2]. En este marco, MongoDB se clasifica típicamente como un sistema **CP** (consistencia y tolerancia a particiones), ya que, al perder contacto el nodo primario de un conjunto de réplicas, el sistema prioriza mantener la coherencia de los datos y puede volver temporalmente no disponibles ciertas operaciones de escritura hasta que se elige un nuevo primario [9]. No obstante, esta clasificación binaria resulta incompleta: como señala Abadi mediante el modelo PACELC, incluso en ausencia de particiones un sistema distribuido debe resolver un compromiso entre latencia y consistencia (*Else*), y MongoDB Atlas permite ajustar este balance mediante niveles configurables de *write concern* y *read preference*, pudiendo optar por lecturas más rápidas sobre nodos secundarios con consistencia eventual, o por lecturas estrictamente consistentes sobre el nodo primario [3]. Esta flexibilidad diferencia a MongoDB Atlas de motores AP puros, como Cassandra, orientados de forma predeterminada hacia la disponibilidad continua incluso a costa de la coherencia inmediata [5].

IV. CASO DE USO REAL DOCUMENTADO

Un caso documentado oficialmente por MongoDB corresponde a Nextar, empresa brasileña proveedora de soluciones de punto de venta (POS) para pequeños comercios. Ante el cierre de establecimientos físicos durante la pandemia de COVID-19, la compañía necesitó habilitar rápidamente tiendas en línea para sus clientes y migró su almacenamiento de datos, originalmente local, hacia MongoDB Atlas desplegado sobre Amazon Web Services y Google Cloud Platform. La naturaleza multicloud de Atlas resultó clave para esa migración progresiva, mientras que Atlas Search se integró directamente sobre los datos ya alojados en el clúster para ofrecer búsqueda y agregación de pedidos sin necesidad de sincronizar un motor de búsqueda externo. Con apoyo del servicio profesional de MongoDB, el equipo de Nextar también identificó consultas ineficientes mediante herramientas de diagnóstico de rendimiento y aplicó estrategias de indexado, lo que permitió sostener cientos de miles de productos ofrecidos por múltiples comercios a través de tiendas en línea alojadas por la propia empresa [10]. Este caso ilustra cómo la combinación de escalado elástico, distribución multicloud y búsqueda integrada de Atlas resuelve necesidades reales de continuidad de negocio bajo condiciones de crecimiento repentino de demanda.

V. PERTINENCIA PARA EL PROYECTO PROPIO

Dentro del contexto de la asignatura de Aplicaciones Distribuidas, un proyecto académico que involucre múltiples fuentes de datos heterogéneas, esquemas cambiantes durante etapas tempranas de desarrollo y necesidad de escalado progresivo encuentra en MongoDB Atlas una opción razonable frente a un motor relacional tradicional. Su modelo de documentos tipo JSON/BSON facilita representar entidades con atributos variables sin someterlas a migraciones de esquema constantes, algo frecuente en proyectos universitarios que evolucionan por iteraciones [6]. Además, el nivel gratuito de Atlas permite desplegar un clúster distribuido real, con réplicas activas y copias de seguridad automáticas, sin incurrir en costos de infraestructura, lo cual resulta idóneo para validar conceptos de tolerancia a fallos y consistencia estudiados en la asignatura antes de trasladar esas decisiones a un motor NewSQL más exigente en configuración, como CockroachDB o YugabyteDB [5]. Su principal limitación, sin embargo, radica en la dependencia de la conectividad a internet y en la necesidad de diseñar cuidadosamente la clave de fragmentación cuando el proyecto contemple volúmenes de datos considerables, aspecto que debe evaluarse desde las primeras etapas de modelado.

VI. CONCLUSIONES

MongoDB Atlas constituye una alternativa NoSQL madura para escenarios distribuidos que priorizan flexibilidad de esquema, escalado horizontal automatizado y alta disponibilidad. Su clasificación como sistema predominantemente CP, matizada por el modelo PACELC y por sus niveles configurables de consistencia, le permite adaptarse a distintos perfiles de aplicación sin exigir una reescritura completa de la arquitectura. El caso de Nextar evidencia que estas capacidades no son meramente teóricas, sino que sostienen operaciones críticas de negocio en entornos reales de alta demanda. Para un proyecto

académico de aplicaciones distribuidas, la curva de aprendizaje reducida y el nivel gratuito de Atlas lo convierten en un punto de partida práctico para experimentar con conceptos de distribución, replicación y consistencia antes de abordar motores NewSQL de mayor complejidad operativa.

REFERENCIAS

- [1] E. A. Brewer, "Towards robust distributed systems (abstract)," in *Proc. 19th Annu. ACM Symp. Principles of Distributed Computing (PODC '00)*, Portland, OR, USA, 2000, p. 7, doi: 10.1145/343477.343502.
- [2] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, Jun. 2002, doi: 10.1145/564585.564601.
- [3] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," *Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012, doi: 10.1109/MC.2012.33.
- [4] A. Davoudian, L. Chen, and M. Liu, "A survey on NoSQL stores," *ACM Comput. Surv.*, vol. 51, no. 2, Art. no. 40, Apr. 2018, doi: 10.1145/3158661.
- [5] L. F. Da Silva and J. V. F. Lima, "An evaluation of relational and NoSQL distributed databases on a low-power cluster," *J. Supercomput.*, vol. 79, pp. 13402–13420, 2023, doi: 10.1007/s11227-023-05166-7.
- [6] H.-J. Kim, E.-J. Ko, Y.-H. Jeon, and K.-H. Lee, "Techniques and guidelines for effective migration from RDBMS to NoSQL," *J. Supercomput.*, vol. 76, no. 10, pp. 7936–7950, 2020, doi: 10.1007/s11227-018-2361-2.
- [7] J. Antas, R. Rocha Silva, and J. Bernardino, "Assessment of SQL and NoSQL systems to store and mine COVID-19 data," *Computers*, vol. 11, no. 2, Art. no. 29, 2022, doi: 10.3390/computers11020029.
- [8] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proc. 1st ACM Symp. Cloud Computing (SoCC '10)*, Indianapolis, IN, USA, 2010, pp. 143–154, doi: 10.1145/1807128.1807152.
- [9] S. Solat, "Sharding distributed databases: A critical review," *arXiv preprint arXiv:2404.04384*, 2024, doi: 10.48550/arXiv.2404.04384.
- [10] MongoDB, Inc., "Cómo utilizar MongoDB Atlas y Atlas Search para ayudar a la modernización digital de las tiendas — Caso Nextar." [En línea]. Disponible: <https://www.mongodb.com/es/solutions/customer-case-studies/nextar>. [Accedido: 8-jul-2026].